



CASE STUDY

MAZE SOLVER BOT

BY

A.Bheeshma Shankar , Chitampally Pranav

160124748093, 160124748099

B.E. CSM-2

IV SEMESTER

PRINCIPLES OF ARTIFICIAL INTELLIGENCE LAB

22CAC06N

ACADEMIC YEAR: 2025-2026

SUBMITTED TO

DR.Y.RAMADEVI

MS.T.RAMYA

MR.KURIKYALA PREMCHAND

MS.P.VENKATA LAKSHMI DURGA

3



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY (A) GANDIPET,

HYDERABAD - 500075

1. Aim

To implement the A* Search Algorithm in Python using Pygame to find the shortest path between a start node and a goal node in a 2D grid with obstacles, and to visualize the process using an interactive graphical interface.

2. Objectives

- To understand the concept of heuristic-based search
 - To implement the A* algorithm for shortest path finding
 - To use Manhattan distance as a heuristic function
 - To create an interactive grid using Pygame
 - To allow user-controlled placement of start, goal, and obstacles
 - To visualize the pathfinding process step-by-step
 - To display the final path and number of steps
 - To analyze the efficiency and performance of the A* algorithm
-

3. Introduction

Pathfinding algorithms are widely used in Artificial Intelligence for solving navigation and routing problems. They are essential in applications such as robotics, video games, GPS systems, and network routing.

The A* algorithm is one of the most efficient and popular pathfinding algorithms due to its ability to combine both actual distance and estimated distance to reach the goal. This makes it significantly faster than uninformed algorithms such as Breadth-First Search (BFS).

In this project, the A* algorithm is implemented in a 2D grid environment using Pygame. The graphical interface allows users to interactively create a grid, place obstacles, and visually observe how the algorithm explores nodes and determines the optimal path.

4. Theory

4.1 A* Algorithm

A* is an informed search algorithm that combines the strengths of:

- Dijkstra's Algorithm (guarantees shortest path)

- Greedy Best-First Search (uses heuristic for faster search)

It evaluates each node using the cost function:

$$f(n)=g(n)+h(n)$$

Where:

- **$g(n)$** = cost from start node to current node
- **$h(n)$** = estimated cost from current node to goal
- **$f(n)$** = total estimated cost of the path

The algorithm always selects the node with the lowest $f(n)$ value, ensuring that it moves toward the goal efficiently while still guaranteeing the shortest path.

4.2 Heuristic Function

The heuristic used in this implementation is the **Manhattan Distance**:

$$h(n)=|x1-x2|+|y1-y2|$$

This heuristic is suitable because:

- Movement is restricted to four directions:
 - Up
 - Down
 - Left
 - Right
 - It is **admissible**, meaning it never overestimates the actual cost
 - It ensures that A* always finds the optimal path
-

4.3 Key Properties of A*

- **Completeness:** Always finds a solution if one exists
 - **Optimality:** Guarantees shortest path with admissible heuristic
 - **Efficiency:** Reduces unnecessary exploration compared to BFS
-

5. System Design

5.1 Grid Representation

- A **10 × 10 grid** is used
- Each cell is represented as a **Node object**
- The grid is displayed using a graphical interface

Each node is assigned a color to represent its state:

Node Type	Color	Description
Start	Green	Starting point
Goal	Red	Destination
Wall	Blue	Obstacle
Path	Yellow	Shortest path
Empty	Dark	Free space

5.2 Data Structures Used

- **Priority Queue (Min Heap):**
Used to store nodes based on their $f(n)$ value and efficiently retrieve the lowest cost node
 - **Dictionary (came_from):**
Keeps track of each node's parent to reconstruct the path
 - **Dictionary (g_score):**
Stores the shortest known distance from the start node to each node
-

6. Algorithm

1. Initialize an open list (priority queue) and insert the start node
2. Set $g(\text{start}) = 0$
3. Repeat until the open list is empty:
 - Select the node with the lowest $f(n)$ value
 - If the selected node is the goal:
 - Reconstruct the path using parent nodes
 - Return the path
 - Otherwise:
 - Explore all valid neighboring nodes (up, down, left, right)
 - Ignore walls and nodes outside grid boundaries
 - Compute:

 $\text{tentative_g} = g(\text{current}) + 1$
 - If this path is better:
 - Update $g(\text{neighbor})$
 - Compute: $f = g + h$
 - Add neighbor to open list
 - Store parent node
4. If the goal is not reached, return failure

6.2 Pseudocode

```
FUNCTION A_STAR(grid, start, goal):
    open_set ← priority queue
    push(start)
    g_score[start] = 0
    came_from = {}
    while open_set is not empty:
        current ← node with lowest f value
        if current == goal:
            return reconstruct_path(came_from, current)
        for each neighbor of current:
            if neighbor is valid and not a wall:
                tentative_g = g_score[current] + 1
                if neighbor not visited OR tentative_g <
g_score[neighbor]:
                    came_from[neighbor] = current
                    g_score[neighbor] = tentative_g
                    f_score = tentative_g + heuristic(neighbor, goal)
                    push (f_score, neighbor) into open_set
    return No Path Found
```

7. Complexity Analysis

The performance of the A* Search Algorithm depends on the number of nodes explored in the grid and the efficiency of the priority queue operations.

7.1 Time Complexity

- Let grid size = $N \times M$
- Total nodes = $V = N \times M$
- Each node is inserted and removed from the priority queue

Time Complexity:

$$O(V \log V) = O(N \times M \log (N \times M))$$

7.2 Space Complexity

- Stores:
 - g_score (cost of each node)
 - came_from (parent tracking)
 - open_set (priority queue)

Space Complexity:

$$O(N \times M)$$

CODE

```
import pygame
import heapq

# ----- SETTINGS -----
WIDTH = 500
HEIGHT = 650
ROWS = 10
CELL = WIDTH // ROWS

# Colors
BG = (10, 10, 30)
GRID = (40, 40, 80)
NEON_GREEN = (57, 255, 20)
NEON_RED = (255, 20, 147)
NEON_BLUE = (0, 200, 255)
NEON_YELLOW = (255, 255, 0)
WHITE = (240, 240, 240)
UI_BG = (20, 20, 50)
pygame.init()
win = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Neon Treasure Hunt")
font = pygame.font.SysFont("Consolas", 24)

# ----- NODE -----
class Node:
    def __init__(self, r, c):
        self.r = r
        self.c = c
        self.color = BG
    def draw(self):
        pygame.draw.rect(win, self.color, (self.c*CELL, self.r*CELL, CELL, CELL))
        pygame.draw.rect(win, GRID, (self.c*CELL, self.r*CELL, CELL, CELL), 1)
    def pos(self):
        return (self.r, self.c)

# ----- GRID -----
def make_grid():
    return [[Node(i,j) for j in range(ROWS)] for i in range(ROWS)]

def draw(grid, msg="", steps=0):
    win.fill(BG)
```

```

for row in grid:
    for node in row:
        node.draw()

# UI Panel
pygame.draw.rect(win, UI_BG, (0, 500, WIDTH, 150))

# Buttons
pygame.draw.rect(win, NEON_BLUE, (50, 520, 150, 50))
pygame.draw.rect(win, NEON_RED, (300, 520, 150, 50))
win.blit(font.render("START", True, BG), (85, 530))
win.blit(font.render("RESET", True, BG), (335, 530))

# Message
if msg:
    color = NEON_GREEN if "GOAL" in msg else WHITE
    text = font.render(msg, True, color)
    win.blit(text, (WIDTH//2 - text.get_width()//2, 580))

# Steps
if steps > 0:
    step_text = font.render(f"Steps: {steps}", True, NEON_YELLOW)
    win.blit(step_text, (WIDTH//2 - step_text.get_width()//2, 610))
pygame.display.update()

# ----- A* -----
def heuristic(a,b):
    return abs(a[0]-b[0]) + abs(a[1]-b[1])

def get_neighbors(node, grid):
    res=[]
    r,c=node.pos()
    for dx,dy in [(0,1),(1,0),(0,-1),(-1,0)]:
        nr,nc=r+dx,c+dy
        if 0<=nr<ROWS and 0<=nc<ROWS:
            if grid[nr][nc].color != NEON_BLUE:
                res.append(grid[nr][nc])
    return res

def astar(grid, start, end):
    open_list=[]
    count=0
    heapq.heappush(open_list,(0,count,start))

    came={}

```

```

g={start:0}
while open_list:
    _,_,current=heapq.heappop(open_list)
    if current==end:
        path=[]
        while current in came:
            path.append(current)
            current=came[current]
        return path[::-1]

    for n in get_neighbors(current,grid):
        temp=g[current]+1
        if n not in g or temp<g[n]:
            g[n]=temp
            f=temp+heuristic(n.pos(),end.pos())
            count+=1
            heapq.heappush(open_list,(f,count,n))
            came[n]=current

return None

def main():
    grid=make_grid()
    start=None
    end=None
    msg=""
    steps=0
    run=True
    while run:
        draw(grid,msg,steps)
        for e in pygame.event.get():
            if e.type==pygame.QUIT:
                run=False

        # LEFT CLICK
        if e.type == pygame.MOUSEBUTTONDOWN and e.button == 1:
            x,y = pygame.mouse.get_pos()

        # START button
        if 50<=x<=200 and 520<=y<=570 and start and end:
            msg="Searching..."
            steps=0
            path=astar(grid,start,end)
            if path:
                for n in path:

```

```

        if n!=start and n!=end:
            n.color=NEON_YELLOW
            steps+=1
            draw(grid,msg,steps)
            pygame.time.delay(70)
            msg=" GOAL REACHED!"
        else:
            msg=" No Path Found"

# RESET button
elif 300<=x<=450 and 520<=y<=570:
    grid=make_grid()
    start=None
    end=None
    msg=""
    steps=0

# GRID CLICK
elif y < WIDTH:
    r,c=y//CELL,x//CELL
    node=grid[r][c]
    if not start:
        start=node
        node.color=NEON_GREEN
    elif not end:
        end=node
        node.color=NEON_RED
    else:
        node.color=NEON_BLUE

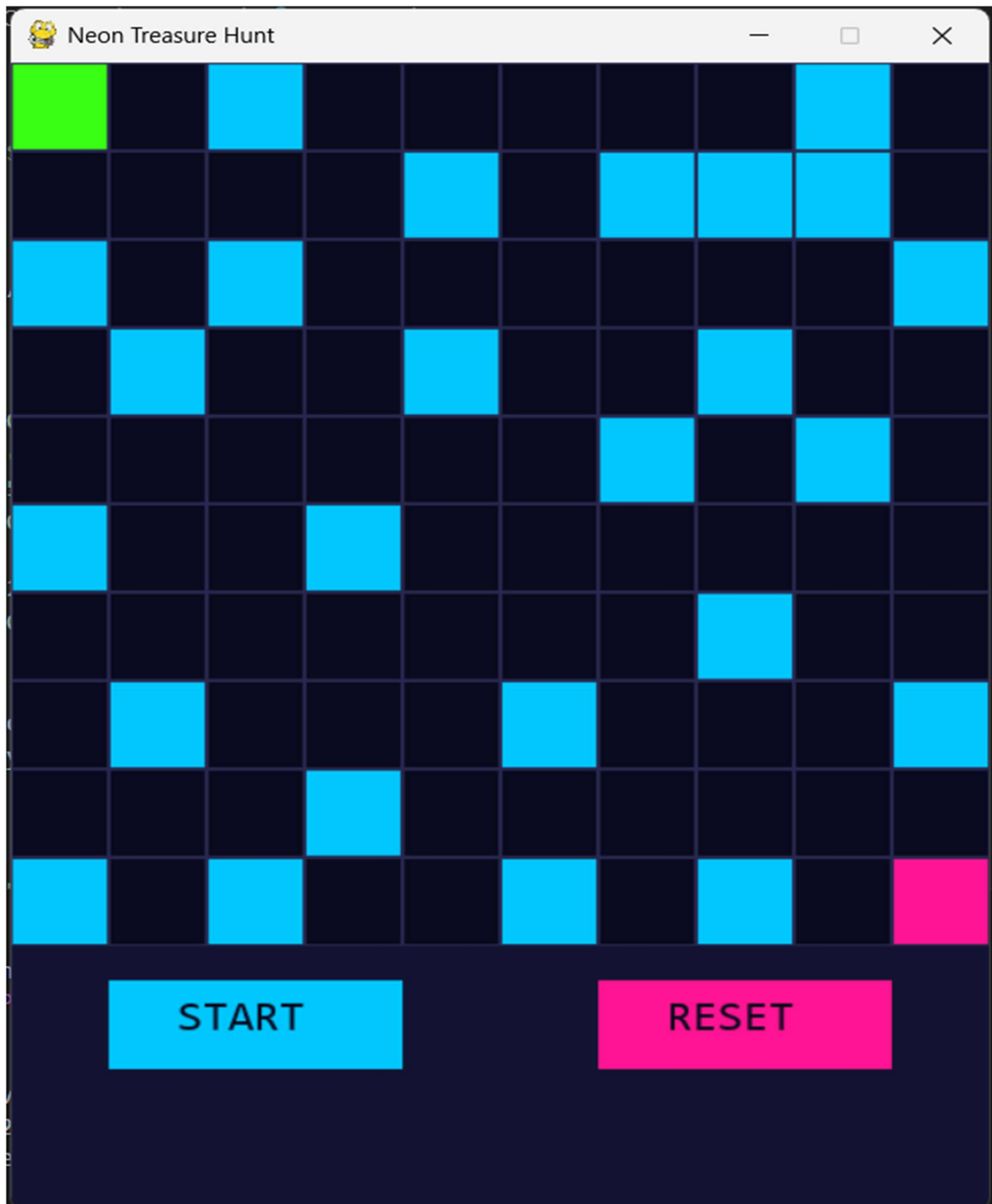
# RIGHT CLICK
if e.type == pygame.MOUSEBUTTONDOWN and e.button == 3:
    x,y=pygame.mouse.get_pos()
    if y < WIDTH:
        r,c=y//CELL,x//CELL
        node=grid[r][c]
        node.color=BG
        if node==start: start=None
        if node==end: end=None

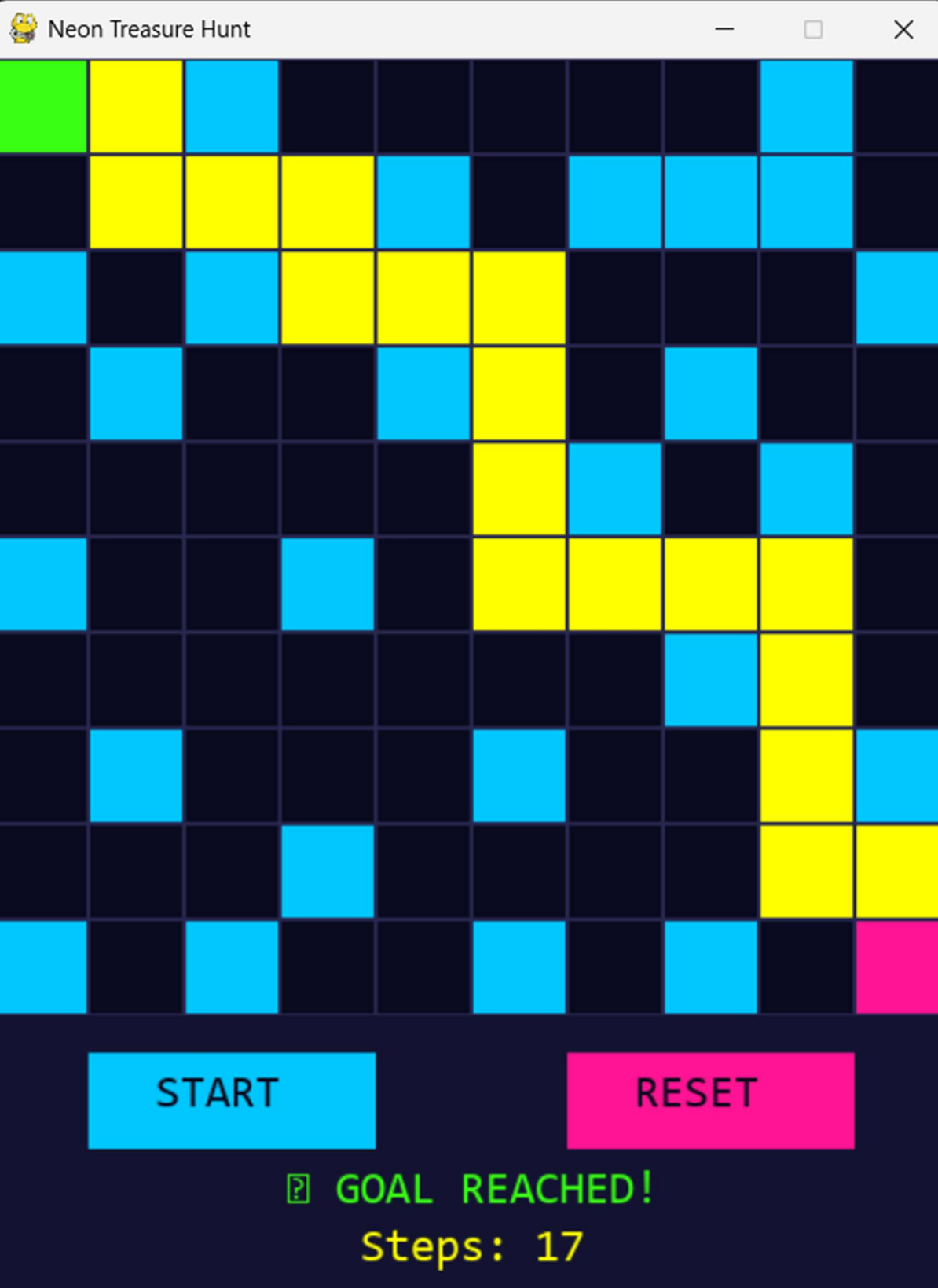
pygame.quit()

# ----- RUN -----
if __name__ == "__main__":
    print("Game is starting...")
    main()

```

Output





8. Conclusion

The A* algorithm successfully computes the shortest path using heuristic guidance. The integration with Pygame provides an interactive and visually appealing way to understand the algorithm. The project demonstrates both algorithmic efficiency and user-friendly design, making it a valuable learning tool in Artificial Intelligence and game development.